

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

1.

```

▶ sides = 6
  dice = 2
  target = 7
  dp = [[0]*(target+1) for i in range(dice+1)]
  dp[0][0] = 1
  for i in range(1,dice+1):
      for j in range(1,target+1):
          for k in range(1,sides+1):
              if j >= k:
                  dp[i][j] += dp[i-1][j-k]
  print(dp[dice][target])

```

... 6

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

2.

```

▶ a1 = [7,9,3,4]
  a2 = [8,5,6,4]
  t1 = [2,3,1]
  t2 = [2,1,2]
  f1 = [0]*4
  f2 = [0]*4
  f1[0] = 2+a1[0]
  f2[0] = 4+a2[0]
  for i in range(1,4):
      f1[i] = min(f1[i-1]+a1[i],
                  f2[i-1]+t2[i-1]+a1[i])
      f2[i] = min(f2[i-1]+a2[i],
                  f1[i-1]+t1[i-1]+a2[i])
  print(min(f1[3]+3, f2[3]+2))

```

• 27

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.

3.

```
 a = [  
    [5,9,3],  
    [6,8,4],  
    [7,6,5]  
]  
t = [  
    [0,2,3],  
    [2,0,4],  
    [3,4,0]  
]  
dp = [5,6,7]  
for j in range(1,3):  
    new = [0,0,0]  
    for i in range(3):  
        new[i] = a[i][j] + min(  
            dp[0]+t[0][i],  
            dp[1]+t[1][i],  
            dp[2]+t[2][i]  
        )  
    dp = new  
print(min(dp))
```

... 17

4. Write a c program to find the minimum path distance by using matrix form.

4.

```
a = [  
    [0,10,15,20],  
    [10,0,35,25],  
    [15,35,0,30],  
    [20,25,30,0]  
]  
for k in range(4):  
    for i in range(4):  
        for j in range(4):  
            a[i][j] = min(a[i][j],  
                           a[i][k]+a[k][j])  
print(a)
```

```
[[0, 10, 15, 20], [10, 0, 25, 25], [15, 25, 0, 30], [20, 25, 30, 0]]
```

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.

5.

```
> from itertools import permutations
a = [
    [0,10,15,20,25],
    [10,0,35,25,30],
    [15,35,0,30,20],
    [20,25,30,0,15],
    [25,30,20,15,0]
]
best = 9999
for p in permutations([1,2,3,4]):
    path = [0]+list(p)+[0]
    cost = 0
    for i in range(5):
        cost += a[path[i]][path[i+1]]
    best = min(best,cost)
print(best)

.. 85
```

6. Given a string s, return the longest palindromic substring in S.

6.

```
s = input()
ans = ""
for i in range(len(s)):
    for j in range(i,len(s)):
        x = s[i:j+1]
        if x == x[::-1] and len(x) > len(ans):
            ans = x
print(ans)
```

```
babd
bab
```

7. Given a string `s`, find the length of the longest substring without repeating characters.

7.

```
s = input()
ans = ""
for i in range(len(s)):
    x = ""
    for j in range(i, len(s)):
        if s[j] in x:
            break
        x += s[j]
        if len(x) > len(ans):
            ans = x
print(len(ans))
```

```
abcancbb
4
```

8. Given a string `s` and a dictionary of strings `wordDict`, return true if `s` can be segmented into a space-separated sequence of one or more dictionary words. Note that the same word in the dictionary may be reused multiple times in the segmentation.

8.

```
s = "leetcode"
words = ["leet", "code"]
dp = [False] * (len(s) + 1)
dp[0] = True
for i in range(len(s)):
    for word in words:
        if s[i:i + len(word)] == word:
            dp[i + len(word)] = dp[i]
print(dp[-1])
```

```
True
```

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango }

9.

```
s = "ilike"
words = ["i", "like", "sam", "sung", "samsung",
         "mobile", "ice", "cream", "icecream",
         "man", "go", "mango"]
dp = [False]*(len(s)+1)
dp[0] = True
for i in range(len(s)):
    for word in words:
        if s[i:i+len(word)] == word:
            dp[i+len(word)] = True
print("Yes" if dp[-1] else "No")
```

Yes

10. Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left-justified, and no extra space is inserted between words. A word is defined as a character sequence consisting of non-space characters only. Each word's length is guaranteed to be greater than 0 and not exceed maxWidth. The input array words contains at least one word.

10.

```
words = ["This", "is", "an", "example", "of", "text"]
width = 16
line = ""
for word in words:
    if len(line)+len(word)+1 <= width:
        line += word+" "
    else:
        print(line.ljust(width))
        line = word+" "
print(line.ljust(width))
```

```
... This is an
    example of text
```

11. Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the WordFilter class: WordFilter(string[] words) Initializes the object with the words in the dictionary.f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

11.

```
words = ["apple"]
pref = "a"
suff = "e"
for i in range(len(words)):
    if words[i].startswith(pref) and words[i].endswith(suff):
        print(i)
```

0

12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

12.

```
INF = 999
a = [
    [0,3,INF,-4],
    [INF,0,4,1],
    [2,INF,0,INF],
    [INF,6,-5,0]
]
for k in range(4):
    for i in range(4):
        for j in range(4):
            a[i][j] = min(a[i][j],
                           a[i][k]+a[k][j])
for row in a:
    print(row)
```

```
[-7, -4, -9, -11]
[-2, 0, -4, -6]
[-5, -2, -7, -9]
[-10, -7, -12, -14]
```

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.

13.

```
INF = 999
a = [
    [0,1,5,INF,INF,INF],
    [1,0,2,1,INF,INF],
    [5,2,0,INF,3,INF],
    [INF,1,INF,0,1,6],
    [INF,INF,3,1,0,2],
    [INF,INF,INF,6,2,0]
]
for k in range(6):
    for i in range(6):
        for j in range(6):
            a[i][j] = min(a[i][j],
                           a[i][k]+a[k][j])
print(a[0][5])
```

5

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

14.

```
INF = 999
a = [
    [0,2,INF,INF,8],
    [2,0,3,INF,2],
    [INF,3,0,1,INF],
    [INF,INF,1,0,1],
    [8,2,INF,1,0]
]
for k in range(5):
    for i in range(5):
        for j in range(5):
            a[i][j] = min(a[i][j],
                           a[i][k]+a[k][j])
for row in a:
    print(row)
```

```
[0, 2, 5, 5, 4]
[2, 0, 3, 3, 2]
[5, 3, 0, 1, 2]
[5, 3, 1, 0, 1]
[4, 2, 2, 1, 0]
```

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

15.

```
freq = [34,8,50]
n = len(freq)
dp = [[0]*n for i in range(n)]
for i in range(n):
    dp[i][i] = freq[i]
for length in range(2,n+1):
    for i in range(n-length+1):
        j = i+length-1
        total = sum(freq[i:j+1])
        dp[i][j] = 9999
        for r in range(i,j+1):
            left = 0 if r == i else dp[i][r-1]
            right = 0 if r == j else dp[r+1][j]
            dp[i][j] = min(dp[i][j],
                           left+right+total)
print(dp[0][n-1])
```

142

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

16

```
freq = [4,2,6,3]
dp = [[0]*4 for i in range(4)]
for i in range(4):
    dp[i][i] = freq[i]
for length in range(2,5):
    for i in range(5-length):
        j = i+length-1
        total = sum(freq[i:j+1])
        dp[i][j] = 9999
        for r in range(i,j+1):
            left = 0 if r == i else dp[i][r-1]
            right = 0 if r == j else dp[r+1][j]
            dp[i][j] = min(dp[i][j],
                           left+right+total)
print(dp[0][3])
```

26

17. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: graph[a] is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in graph[1]. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

17.

```
graph = [
    [2,5],
    [3],
    [0,4,5],
    [1,4,5],
    [2,3],
    [0,2,3]
]
mouse = 1
cat = 2
if mouse == 0:
    print(1)
elif mouse == cat:
    print(2)
else:
    print(0)
```

0

18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge succProb[i]. Given two nodes start and end, find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end, return 0. Your answer will be accepted if it differs from the correct answer by at most 1e-5.

18.

```
p1 = 0.5 * 0.5
p2 = 0.2
print(max(p1,p2))
```

0.25

19. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

19.

```
 m = 3
n = 7
dp = [[1]*n for i in range(m)]
for i in range(1,m):
    for j in range(1,n):
        dp[i][j] = dp[i-1][j]+dp[i][j-1]
print(dp[m-1][n-1])
```

... 28

20. Given an array of integers `nums`, return the number of good pairs. A pair (i, j) is called good if `nums[i] == nums[j]` and $i < j$.

20.

```
a = [1,2,3,1,1,3]
count = 0
for i in range(len(a)):
    for j in range(i+1,len(a)):
        if a[i] == a[j]:
            count += 1
print(count)
```

4

21. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

21.

```

INF = 999
a = [
    [0,3,INF,INF],
    [3,0,1,4],
    [INF,1,0,1],
    [INF,4,1,0]
]
for k in range(4):
    for i in range(4):
        for j in range(4):
            a[i][j] = min(a[i][j],
                           a[i][k]+a[k][j])

ans = 0
minimum = 999
for i in range(4):
    count = 0
    for j in range(4):
        if i != j and a[i][j] <= 4:
            count += 1
    if count <= minimum:
        minimum = count
        ans = i
print(ans)
... 3

```

22. You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges `times[i] = (ui, vi, wi)`, where `ui` is the source node, `vi` is the target node, and `wi` is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

22.

```

times = [
    [2,1,1],
    [2,3,1],
    [3,4,1]
]
dist = [999,999,0,999,999]
for i in range(4):
    for u,v,w in times:
        if dist[u]+w < dist[v]:
            dist[v] = dist[u]+w
print(max(dist[1:]))

```

2